Software Requirements Specification (SRS)

Smart Corridor Parking Finder Application (Web + Mobile)

Backend: Flask
Frontend: React (Primary) + Vue (Optional Components)
Database: SQLite (Default) → PostgreSQL (Future)

1. Introduction

1.1 Purpose

This document defines the complete functional and non-functional requirements for the Smart Parking Finder Application, including:

- Parking discovery via interactive map
- Search and nearby parking functionality
- User activity tracking and analytics
- Admin dashboard reporting
- Database persistence using SQLite (extensible to PostgreSQL)

1.2 Scope

The system shall:

1. Display parking facilities loaded from a GeoJSON file.
2. Provide interactive map functionality.
3. Support search and nearby parking.
4. Track user activities and persist them in a database.
5. Provide an admin dashboard for usage reporting.
6. Be mobile-first and lightweight.
7. Be designed for future integration and database migration.

2. Overall System Description

2.1 System Architecture

```
Frontend (React / Vue)
        ↓
Flask REST API
        ↓
SQLite Database (Users & Activities)
```

```
        ↓
GeoJSON File (Parking Data)
```

Future Phase:

```
Flask API
        ↓
PostgreSQL + PostGIS
```

2.2 User Classes

2.2.1 Public User

- View map
- Search parking
- Use nearby function
- Identify facilities

2.2.2 Admin User

- View dashboard analytics
- Access usage reports
- View activity breakdown

3. Functional Requirements

3.1 Parking Data Module

FR-01: GeoJSON Loading

- The system shall load parking facilities from a GeoJSON file at application startup.
- The data shall be cached in memory.
- The system shall validate that:
    - Geometry type is Point.
    - Coordinates are WGS84 (EPSG:4326).
    - Required properties exist (id, name, address).

FR-02: Retrieve All Parking Facilities

Endpoint:

```
GET /api/parking
```

- Return all parking facilities in GeoJSON format.

- Response must follow FeatureCollection schema.

FR-03: Search Parking

Endpoint:

```
GET /api/parking?search=<keyword>
```

Requirements:

- Case-insensitive search.
- Match against:
  - name
  - address
- Return filtered FeatureCollection.

FR-04: Nearby Parking

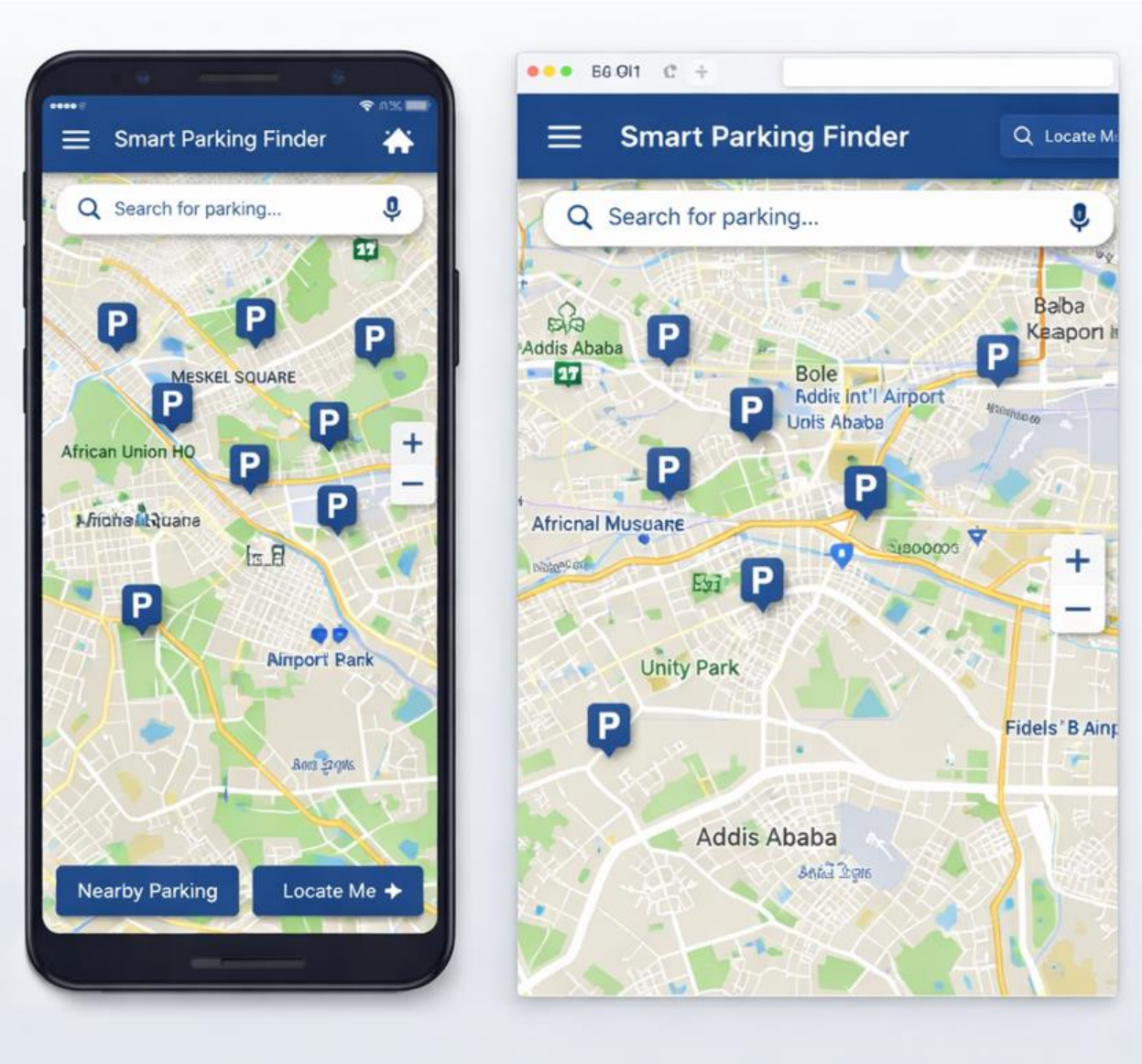
Endpoint:

```
GET /api/parking/nearby?lat=<float>&lon=<float>&radius=<meters>
```

Requirements:

- Use Haversine formula for distance calculation.
- Return only features within radius.
- Include calculated distance in response properties.
- Sort results by ascending distance.

3.2 Map Interface Requirements



FR-05: Map Display

- Display interactive Leaflet map.
- Configurable default center and zoom.
- Use OpenStreetMap as base layer.

FR-06: Map Controls

- Zoom in/out controls.
- Mouse wheel zoom.
- Touch pinch zoom.

- Layer toggle for parking layer.

FR-07: Feature Identify

- Clicking a parking marker shall display popup.
- Popup shall show:
  - Name
  - Address
  - Capacity (if available)
  - Status (if available)

3.3 User Tracking & Analytics Module

FR-08: User Identification

- System shall generate a UUID for each user.
- UUID stored in browser localStorage.
- UUID sent with every tracked activity.

FR-09: Activity Logging

Endpoint:

```
POST /api/activity
```

The system shall log the following activities:

Activity Code	Description
APP_LOAD	App opened
SEARCH	Search performed
NEARBY_SEARCH	Nearby function used
IDENTIFY	Parking clicked
GEOLOCATION	User allowed location

Each activity shall store:

- user_uuid
- activity_type
- timestamp
- latitude (if available)
- longitude (if available)
- parking_id (if applicable)
- search_query (if applicable)

FR-10: Database Persistence

- Default database: SQLite.
- Use SQLAlchemy ORM.
- System must avoid database-specific SQL.
- Design must support migration to PostgreSQL.

3.4 Admin Dashboard Module

FR-11: Admin Authentication (MVP)

- Admin routes shall require static token from environment variable.
- Token validated on each request.

FR-12: Total Users Endpoint

```
GET /api/admin/total-users
```

- Count distinct user_uuid.
- Return JSON summary.

FR-13: Activity Summary Endpoint

```
GET /api/admin/activity-summary
```

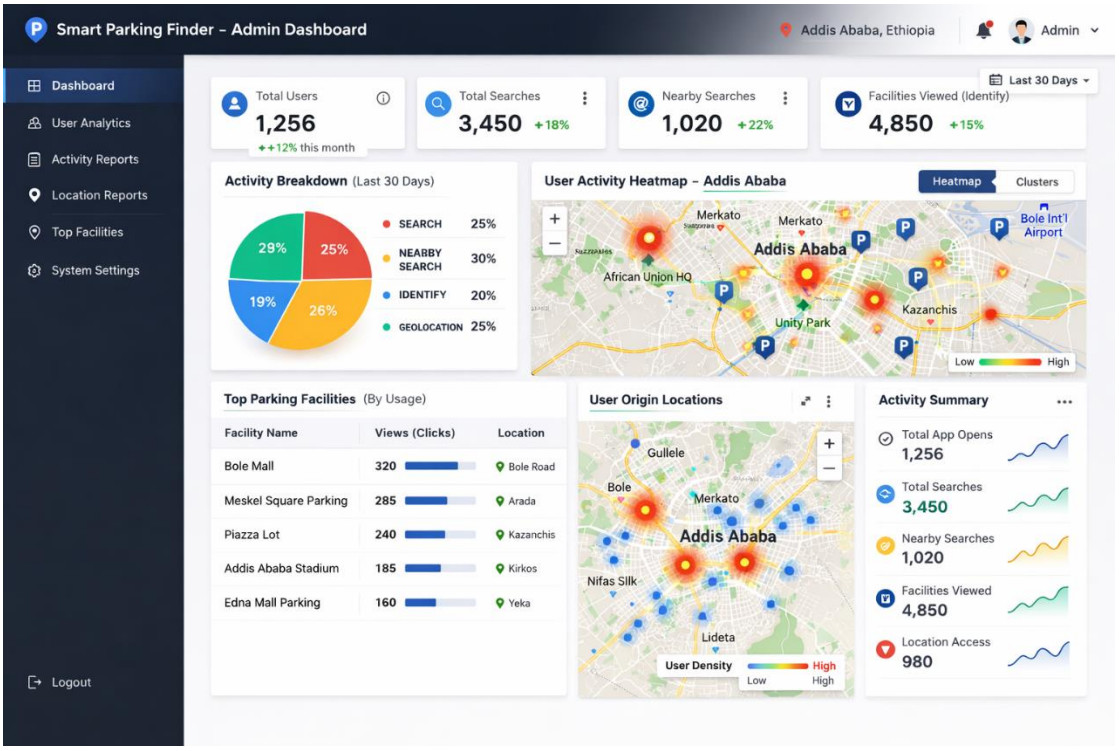
- Return activity count grouped by activity_type.

FR-14: Top Facilities Endpoint

```
GET /api/admin/top-facilities
```

- Return facilities ranked by IDENTIFY activity.
- Limit default result to Top 10.

FR-15: Dashboard UI



The admin interface shall display:

1. Total number of users.
2. Activity breakdown chart (bar or pie).
3. Top facilities table.
4. Total searches, nearby searches, and geolocation uses.
5. Dashboard must be mobile responsive.

4. Non-Functional Requirements

4.1 Performance

- Initial load < 3 seconds (4G).
- API response < 500ms for dataset under 5000 records.
- Activity logging must not block UI interaction.

4.2 Usability

- Mobile-first design.
- Touch-friendly controls.
- Minimal interface clutter.
- Clear error messages.

4.3 Security

- Validate all query parameters.
- Sanitize inputs.
- Protect admin endpoints with token.
- Enforce HTTPS in production.
- Configure CORS properly.

4.4 Scalability

- Stateless API design.
- ORM-based data access.
- Prepared for PostgreSQL migration.
- Modular service architecture.

4.5 Maintainability

- Clean folder structure.
- Separation of concerns:
    - Routes
    - Services
    - Models
    - Utilities
- Environment-based configuration (.env support).

5. Data Model Requirements

5.1 Users Table

Fields:

- id (Primary Key)
- user_uuid (Unique)
- first_seen
- last_seen

5.2 User Activities Table

Fields:

- id (Primary Key)
- user_uuid (Foreign key or indexed)
- activity_type
- parking_id (nullable)
- search_query (nullable)
- latitude (nullable)
- longitude (nullable)
- timestamp

Indexes:

- Index on user_uuid
- Index on activity_type
- Index on parking_id

6. Deployment Requirements

- Dockerized backend.
- Static frontend build.
- Nginx reverse proxy.
- Environment configuration support.
- Logging enabled.
- Production-ready error handling.

7. Future Enhancement Requirements (Out of Scope)

- Real-time parking availability.
- Smart sensor integration.
- Payment gateway integration.
- User authentication system.
- Role-based access control.
- PostgreSQL + PostGIS spatial migration.
- Heatmap visualization for user location analytics.

8. Acceptance Criteria

The system shall be accepted when:

1. Parking data loads correctly from GeoJSON.

2. Map interaction works on desktop and mobile.
3. Search returns correct results.
4. Nearby function returns correct filtered results.
5. All user activities are logged in SQLite.
6. Admin dashboard displays:
    - Total users
    - Activity breakdown
    - Top facilities
7. System runs successfully in Docker.
8. ORM migration to PostgreSQL is structurally validated.